

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 10. List

Lecture: 103.Indexing and Slicing - Write

Section 1: Lecture Summary

This lecture covers how to **modify Python lists using indexing and slicing**. It builds on the previous knowledge of reading list data via indexing and slicing and focuses now on writing or updating elements in a list. The instructor explains how individual elements can be changed by using an index, including negative indices, and how nested lists can be created by assigning a list as an element. Then, the lecture explores slicing syntax ``(start:stop:step)`` for modifying parts of a list, emphasizing different rules depending on whether the step value is provided. Practical examples show inserting single or multiple elements at specific positions using slicing assignment, including how elements are shifted when inserting. The lecture also discusses the important rule that when slice assignment includes a step, the number of replacement elements must exactly match the number of elements in the slice. Negative steps are explained for reversing a list's elements in-place. Throughout, the instructor stresses practicing these operations for mastery, since slicing and indexing are widely used in Python for lists, strings, tuples, and other ordered data types.

Section 2: Key Concepts and Explanations

1. Indexing for Writing / Modifying Lists

- Indexing accesses a single element at a specified position.
- Syntax: ``list[index] = value``
- Can use positive or negative indices (e.g., ``-1`` for last element).
- You can replace a single element with any data type, including another list, thereby creating nested lists.

2. Slicing for Writing / Modifying Lists

- Slicing syntax: ``list[start:stop:step]``
- Used to modify multiple elements or insert new elements.
- When only start and stop are specified (i.e., step omitted):

- You can assign **any number of elements**, not limited to the number of elements being replaced.
- If the number of assigned elements is greater than the slice length, elements after the slice shift right.
- If fewer assigned, elements in the slice are removed and replaced by fewer elements.
- When step is specified along with start and stop:
 - The number of elements in the assignment **must exactly match** the number of elements selected by the slice.
 - Otherwise, Python raises an error.

3. Special Cases and Rules

- Assigning to a slice with ``start == stop`` (e.g., `l[0:0] = [...]`) inserts elements at that index without replacing existing elements.
- Assigning outside the list's range with slice insertion, e.g., `l[9:9] = [10]`, appends the element at the end (like ``append``).
- Negative step reverses the order of indexing:
 - Slices with ``step = -1`` iterate backward.
 - Assignment must still have the exact number of elements as the slice length.
 - Modifies the list in-place to reverse or partially reverse segments.

4. Nested Lists

- Assigning a list to a single index (e.g., `l[3] = [10, 11]`) replaces the element at that index with another list, creating a nested list.

Section 3: Example Code and Use Cases

Example 1: Modify single element by index

Output: [1, 2, 3, 10, 5]

```
l1 = [1, 2, 3, 4, 5]
l1[3] = 10 # Replace element at index 3 (4th element) with 10
print(l1)
```

Single element at index 3 changed from 4 to 10.

Example 2: Assign a list to a single index (nested list)

Output: [1, 2, 3, [10, 11], 5]

```
l1[3] = [10, 11]
print(l1)
```

Element at index 3 replaced with a list, creating nested list.

Example 3: Insert single element at start using slicing

Output: [10, 1, 2, 3, 4, 5]

```
l1 = [1, 2, 3, 4, 5]
l1[0:0] = [10] # Insert 10 at index 0
print(l1)
```

Example 4: Insert element at index 3 using slicing

Output: [10, 1, 2, 10, 3, 4, 5]

```
l1[3:3] = [10]
print(l1)
```

All elements from index 3 onwards shift right.

Example 5: Insert element out of range (appends)

Output: [10, 1, 2, 10, 3, 4, 5, 10]

```
l1[9:9] = [10]
print(l1)
```

Inserting at index beyond list's length appends element.

Example 6: Replace multiple elements with fewer or more elements (only start and stop)

Output: [1, 10, 5]

Output: [1, 10, 11, 5]

Output: [1, 10, 11, 12, 13, 5]

```
l1 = [1, 2, 3, 4, 5]
l1[1:4] = [10] # Replace elements at indices 1,2,3 (2,3,4) with
single 10
print(l1)
```

```
l1 = [1, 2, 3, 4, 5]
l1[1:4] = [10, 11] # Replace indices 1-3 with two elements
print(l1)
```

```
l1 = [1, 2, 3, 4, 5]
l1[1:4] = [10, 11, 12, 13] # Replace with more elements
print(l1)
```

Example 7: Replace elements with step - exact number of replacements required

Output: [10, 2, 11, 4, 12]

```
l1 = [1, 2, 3, 4, 5]
l1[::2] = [10, 11, 12] # Replace every 2nd element (indices 0,2,4)
print(l1)
```

Must supply exactly 3 elements as slice selects 3 elements.

Example 8: Reverse list elements via slicing assignment with negative step

Output: [10, 11, 12, 13, 14]

```
l1 = [1, 2, 3, 4, 5]
l1[::-1] = [10, 11, 12, 13, 14] # Replace all 5 elements in reverse order
print(l1)
```

Negative step reverses slicing. Assignment must have exactly 5 elements.

Example 9: Reverse partial slice with negative step

Output: [1, 14, 13, 12, 5]

```
l1 = [1, 2, 3, 4, 5]
l1[3:0:-1] = [12, 13, 14] # Replace slice indices 3,2,1 in reverse order
print(l1)
```

Assigns replacement values in reversed order for the slice.

Section 4: Key Takeaways

- **Indexing** (`list[index] = value`) **modifies a single element**, and can assign values of any type, including nested lists.
- **Slicing without step** (`list[start:stop] = [...]`) **allows insertion or replacement with any number of elements**, shifting remainder of list if necessary.
- **Slicing with step** (`list[start:stop:step] = [...]`) **requires the replacement list to match slice length exactly**; otherwise, it raises an error.
- Assigning to a slice with `start == stop` inserts elements at that index without removing any element.

- Assigning to slices that exceed the list length appends elements at the end.
- Negative step in slicing allows **replacement or reversal of list elements in reverse order**, with the same exact element count required for assignment.
- Practice with all forms of slicing and indexing is crucial to mastering list modifications — these concepts apply broadly to other ordered data types like strings and tuples.
- Visualizing list modifications and printing after each operation helps to understand element shifts and replacements clearly.